

B.8. Implement KNN, Naïve Bayes, and SVM classifiers using Python-based ML tools for comparison of their performance on the review sentiment classification dataset.

```
In [6]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from typing import List
import re
from collections import Counter
from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, precision_score, recall_score, f
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

# Convert TF-IDF dictionaries to vectors
from sklearn.feature_extraction import DictVectorizer

#this is just to get data for SVM
from sklearn.datasets import make_blobs

import kagglehub

# Download latest version
path = kagglehub.dataset_download("abhi8923shriv/sentiment-analysis-dataset")

print("Path to dataset files:", path)
```

Path to dataset files: /home/rito/.cache/kagglehub/datasets/abhi8923shriv/sentiment-analysis-dataset/versions/9

K Nearest Neighbor (KNN)

Kth nearest neighbors(KNN) does not require any training In a way the dataset itself is the model. we are going to use the dataset itself and the nearest neighbors the predict which cluster out data belong to by calculating the euclidean distance from out point to the nearest points.

formula for euclidean distance on n dimensions are given below:

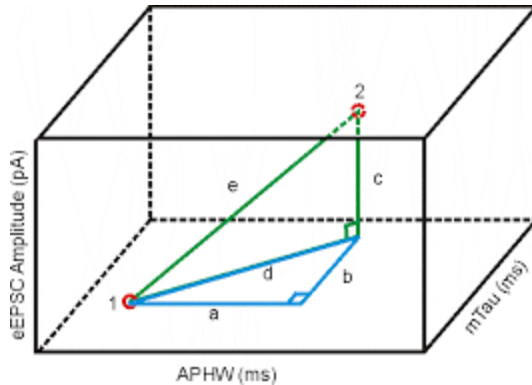
The Euclidean distance in n-dimensions is the straight-line distance between two points, generalized from the Pythagorean theorem: it's the square root of the sum of the squared differences between their corresponding coordinates across all n dimensions. For points $(P_1(x_1, x_2, \dots, x_n))$ and $(P_2(y_1, y_2, \dots, y_n))$, the formula is

$$d(p_1, p_2) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Formula

Points: $(P_1 = (x_1, x_2, \dots, x_n))$ and $(P_2 = (y_1, y_2, \dots, y_n))$.

Distance: $(d(P_1, P_2) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_n - y_n)^2})$



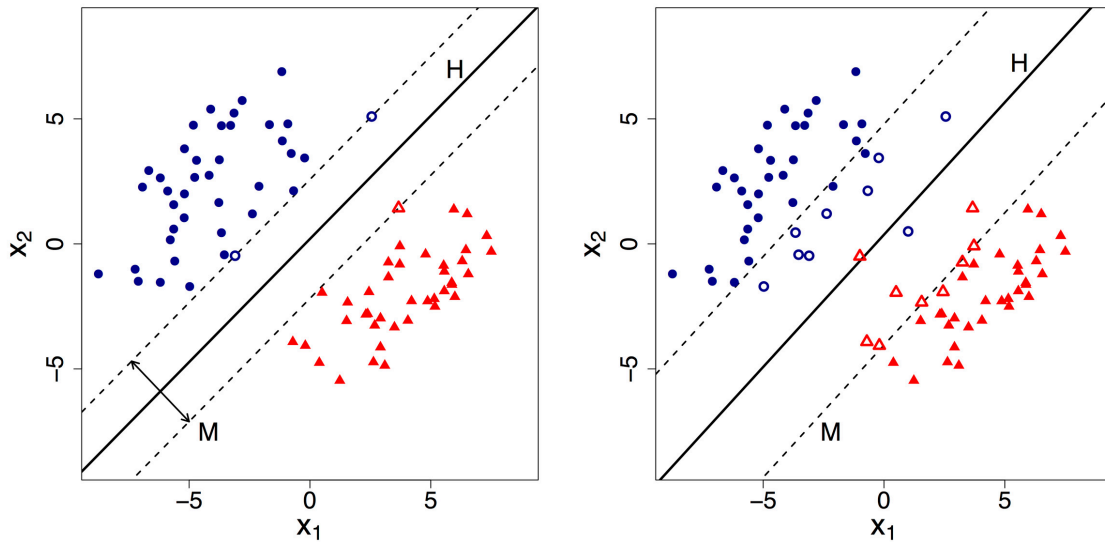
Support Vector Machine (SVM)

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. It tries to find the best boundary known as hyperplane that separates different classes in the data. It is useful when you want to do binary classification like spam vs. not spam or cat vs. dog.

SVM basically finds the boundary between different data classes and while finding tried to find the thickest possible boundary.

Depending on How data inside boundary area is treated SVM can be classified into 2 types

1. Hard margin
2. Soft margin



The above 2 graphs shows 2 different scenarios

1. for Hard Margin we are going to focus on the graph on left
2. for Soft Margin we are going to focus on the graph on the right

1. The Dotted lines are Support vectors
2. The central lines are Separating hyperplane
3. M is the margin

Central Separating Hyperplane equation = $\vec{W} \cdot \vec{X} + b = 0$

• Support Vector = $\vec{W} \cdot \vec{X} + b = 1$

▲ Support Vector = $\vec{W} \cdot \vec{X} + b = -1$

$M = \frac{2}{\|\vec{w}\|}$ where $\|\vec{w}\| = \sqrt{W_1^2 + W_2^2}$ for dataset with n features

$\|\vec{w}\| = \sqrt{W_1^2 + W_2^2 + \dots + W_n^2}$

Hard Margin

For Hard Margin $\Rightarrow$ Algorithm:

$$MAX \frac{2}{\|\vec{w}\|} \Rightarrow MIN \|\vec{W}\|$$

for Hard margin no data can be in the margin region

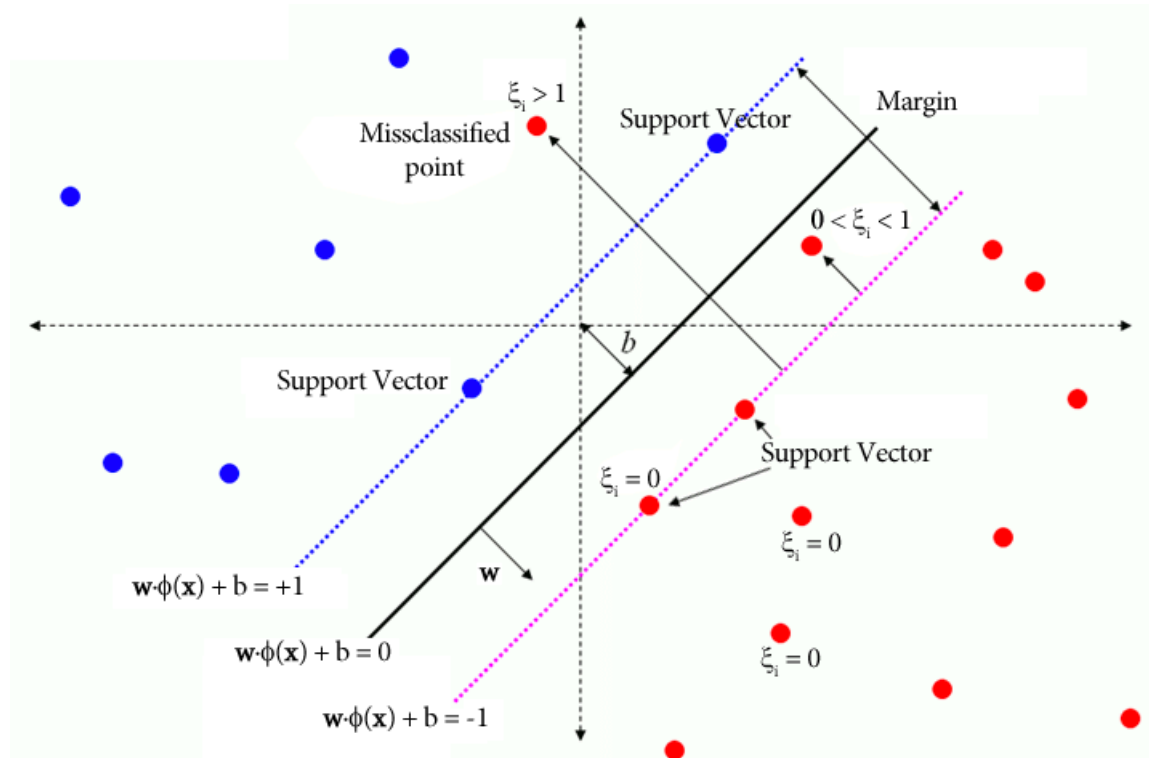
so, based on above we can say they:

$$y^{(i)}(\vec{W} \cdot \vec{X}) \geq 1, y^{(i)} \in \{-1, 1\} \implies \begin{cases} \vec{W} \cdot \vec{X} \geq 1, & y^{(i)} = 1 \\ \vec{W} \cdot \vec{X} \leq -1, & y^{(i)} = -1 \end{cases}$$

Soft Margin

for real life data soft margin is more suited as in real life data there will be overlap in between classes which can be seen from the graph on the right

to consider the data in outside its support vector slack variable is introduced which is denoted as ξ which can be explained through the image below



so based on the above the new constraints are as following

$$y^{(i)}(\vec{W} \cdot \vec{X}) \geq 1 - \xi^{(i)} \implies \begin{cases} \vec{W} \cdot \vec{X} \geq 1 \implies \xi^{(i)} = 0 \\ \vec{W} \cdot \vec{X} \leq -1 \implies \xi^{(i)} = 1 - Y^{(i)}(\vec{W} \cdot \vec{X} + b) \end{cases}$$

from the above we can get the equation to find the value of slack variable for any i th training example which is as following

$$\xi^{(i)} = \text{MAX}(0, 1 - Y^{(i)}(\vec{W} \cdot \vec{X} + b))$$

For Soft Margin $\Rightarrow$ Algorithm:

$$\text{MIN} ||\vec{w}|| \text{ And all } \xi^{(i)}$$

$$\Rightarrow \text{MIN}_{\vec{W}, b} \frac{1}{2} \|\vec{W}\|^2 + C \sum_{i=1}^m \xi^{(i)}$$

for the above equation

1. we are squaring the margin of the weights vector for gradient decent
2. The Constant C controls how much impact the slack variable has over entire equation.

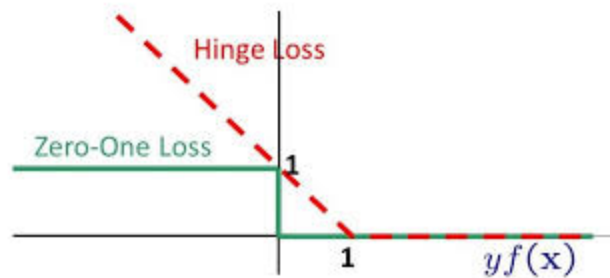
expanding ξ in the above minimization equation

$$\Rightarrow \text{MIN}_{\vec{W}, b} \frac{1}{2} \|\vec{W}\|^2 + C \sum_{i=1}^m (1 - Y^{(i)}(\vec{W} \cdot \vec{X} + b)) \text{ this is main cost function}$$

from the above equation $\sum_{i=1}^m (1 - Y^{(i)}(\vec{W} \cdot \vec{X} + b))$ is called hinge loss

Hinge loss $\Rightarrow$ if a point in data set needs slack then it's going to have a loss and if the point does not need any slack then the loss is zero for that point.

Hinge loss graph is mentioned below



Naïve Bayes

The Naïve Bayes classifier is probabilistic classifier based on applying Bayes theory with strong (naive) independent assumptions between the features.

$$\text{Bayes equation} \Rightarrow P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Bayes theorem states that the probability of an event A from happening given B can be calculated by probability of B given A times probability of A divided by probability of B

1. $P(A|B)$: Posterior Probability – The probability of hypothesis A being true given that evidence B has occurred.
2. $P(B|A)$: Likelihood – The probability of observing evidence B given that hypothesis A is true.

3. $P(A)$: Prior Probability – The initial probability of hypothesis A before seeing any evidence.
4. $P(B)$: Marginal Likelihood (or Evidence) – The total probability of observing the evidence B.

In our case $\Rightarrow P(y|X) = \frac{P(X|y) \cdot P(y)}{P(X)}$

where the y are the class label and X is the feature set vector

In Naïve Bayes we assume that the features are mutually independent

so the new equation after the assumption is as following

$$P(y|X) = \frac{P(x_1|y) \cdot P(x_2|y) \cdot P(x_3|y) \cdot \dots \cdot P(x_n|y) \cdot P(y)}{P(X)}$$

for the above equation $P(y|X)$ is called posterior

for the above equation since the denominator is independent of y and constant for each X so we can Write the equation as following

$$P(y|X) = P(x_1|y) \cdot P(x_2|y) \cdot P(x_3|y) \cdot \dots \cdot P(x_n|y) \cdot P(y)$$

now to determine the class we need to select the one with highest posterior probability so the new equation becomes

$$\hat{y} = \operatorname{argmax}_y P(y) \cdot P(x_1|y) \cdot P(x_2|y) \cdot P(x_3|y) \cdot \dots \cdot P(x_n|y)$$

$$\Rightarrow \hat{y} = \operatorname{argmax}_y P(y) \cdot \prod_{i=1}^n P(x^i|y)$$

Data handling and TD-IDF implementation

```
In [7]: path = path + "/train.csv"
data = pd.read_csv(path, encoding='latin1')
df = pd.DataFrame(data)
```

```
In [8]: sentiment_map = {
    'negative': 0,
    'neutral': 1,
    'positive': 2
}

df['sentiment_label'] = df['sentiment'].map(sentiment_map)
```

```
In [9]: time_map = {
    'morning': [1,0,0],
    'noon': [0,1,0],
    'night': [0,0,1]
```

```
}

df['time_label'] = df['Time of Tweet'].map(time_map)
```

```
In [10]: class TfIdf:
    def __init__(self):
        self.vocab = set()
        self.df_count = {}
        self.idf = {}
        self.N = 0

    def tokenize(self, text):
        if not isinstance(text, str):
            return [] # ignore NaN or non-string values
        cleaned_text = re.sub(r"^\w\s", "", text)
        return cleaned_text.lower().split()

    def build_vocab(self, df, column='message'):
        """Build vocabulary from all documents"""
        self.vocab = set()

        for i in range(len(df)):
            tokens = self.tokenize(df.loc[i, column])
            self.vocab = self.vocab.union(tokens)

        return self.vocab

    def compute_tf(self, tokens):
        """Compute term frequency for a document"""
        if not tokens:
            return {}
        tf = Counter(tokens)
        total = len(tokens)
        return {word: count / total for word, count in tf.items()}

    def compute_df_count(self, df, column='message'):
        """Count document frequency for each word"""
        self.df_count = dict.fromkeys(self.vocab, 0)
        self.N = len(df)

        for i in range(len(df)):
            text = df.loc[i, column]
            tokens = set(self.tokenize(text)) # IMPORTANT: set() to count each word only once
            for word in tokens:
                if word in self.df_count:
                    self.df_count[word] += 1

        return self.df_count

    def compute_idf(self):
        """Compute IDF values for all words in vocabulary"""
        self.idf = {}
        for word, df_val in self.df_count.items():
            self.idf[word] = np.log((self.N + 1) / (df_val + 1)) + 1

        return self.idf
```

```

def compute_tfidf(self, tf):
    """Compute TF-IDF scores given TF values"""
    return {word: tf[word] * self.idf.get(word, 0) for word in tf}

def compute_text_tfidf(self, text):
    """Compute TF-IDF for a single text document"""
    tokens = self.tokenize(text)
    tf = self.compute_tf(tokens)
    return self.compute_tfidf(tf)

def fit(self, df, column='message'):
    """Fit the TF-IDF model on a dataframe"""
    self.build_vocab(df, column)
    self.compute_df_count(df, column)
    self.compute_idf()
    return self

def transform(self, df, column='message'):
    """Transform documents to TF-IDF vectors"""
    tfidf_scores = []
    for i in range(len(df)):
        text = df.loc[i, column]
        tfidf = self.compute_text_tfidf(text)
        tfidf_scores.append(tfidf)
    return tfidf_scores

def fit_transform(self, df, column='message'):
    """Fit and transform in one step"""
    self.fit(df, column)
    return self.transform(df, column)

```

```

In [11]: # Initialize the TF-IDF model
tfidf_model = TfIdf()

# Option 1: Get dictionaries first (like your original code)
df['text_tf_idf'] = tfidf_model.fit_transform(df, column='text')

# Then vectorize (exactly like your original code)
vectorizer = DictVectorizer(sparse=False)
X_all = vectorizer.fit_transform(df['text_tf_idf'].tolist())
y_all = df['sentiment_label'].values

```

```

In [12]: # Use only a sample of your data
sample_size = 5000 # Adjust based on your PC's RAM
df_sample = df.sample(n=sample_size, random_state=42)

# Then convert to vectors
vectorizer = DictVectorizer(sparse=False)
X_sample = vectorizer.fit_transform(df_sample['text_tf_idf'].tolist())
y_sample = df_sample['sentiment_label'].values

# Now split - this should work
X_train, X_test, y_train, y_test = train_test_split(

```

```
X_sample, y_sample, test_size=0.2, random_state=42, stratify=y_sample
)
```

KNN Code

```
In [13]: def euclidean_dist(p,q):
         return np.sqrt(np.sum((np.array(p) - np.array(q))**2))
```

```
In [14]: # 4. Modify your KNN class to work with the data format
class KNN:
    def __init__(self, k=3):
        self.k = k
        self.X_train = None
        self.y_train = None

    def fit(self, X_train, y_train):
        """Store training data"""
        self.X_train = X_train
        self.y_train = y_train

    def predict(self, X_test):
        """Predict labels for test data"""
        predictions = []

        for test_point in X_test:
            # Calculate distances to all training points
            distances = []
            for i, train_point in enumerate(self.X_train):
                distance = euclidean_dist(train_point, test_point)
                distances.append((distance, self.y_train[i]))

            # Sort by distance and get k nearest neighbors
            distances.sort(key=lambda x: x[0])
            k_nearest = distances[:self.k]

            # Get the most common label
            k_labels = [label for _, label in k_nearest]
            prediction = Counter(k_labels).most_common(1)[0][0]
            predictions.append(prediction)

        return np.array(predictions)
```

```
In [15]: k = KNN(k=5)
         k.fit(X_train,y_train)
         y_prid = k.predict(X_test)
```

Confusion Matrix

Confusion matrix is a simple table used to measure how well a classification model is performing. It compares the predictions made by the model with the actual results and shows where the model was right or wrong. This helps you understand where the

model is making mistakes so you can improve it. It breaks down the predictions into four categories

True Positive (TP): The model correctly predicted a positive outcome i.e the actual outcome was positive.

True Negative (TN): The model correctly predicted a negative outcome i.e the actual outcome was negative.

False Positive (FP): The model incorrectly predicted a positive outcome i.e the actual outcome was negative. It is also known as a Type I error.

False Negative (FN): The model incorrectly predicted a negative outcome i.e the actual outcome was positive. It is also known as a Type II error.

1. **Accuracy:** Accuracy shows how many predictions the model got right out of all the predictions

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

2. **Precision:** Precision focus on the quality of the model's positive predictions.

$$Precision = \frac{TP}{TP+FP}$$

3. **Recall:** Recall measures how how good the model is at predicting positives.

$$Recall = \frac{TP}{TP+FN}$$

4. **F1-Score:** F1-score combines precision and recall into a single metric to balance their trade-off. It provides a better sense of a model's overall performance particularly for imbalanced datasets

$$F1-Score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

Confusion Matrix Code

```
In [16]: class CM:
def accuracy(self, test, prid):
    TP = TN = 0
    for i in range(len(test)):
        if test[i] == 1 and prid[i] == 1:
            TP += 1
        elif test[i] == 0 and prid[i] == 0:
```

```

        TN += 1
    return (TP + TN) / len(test)

def precision(self, test, prid):
    TP = FP = 0
    for i in range(len(test)):
        if test[i] == 1 and prid[i] == 1:
            TP += 1
        elif test[i] == 0 and prid[i] == 1:
            FP += 1
    return TP / (TP + FP) if (TP + FP) != 0 else 0

def recall(self, test, prid):
    TP = FN = 0
    for i in range(len(test)):
        if test[i] == 1 and prid[i] == 1:
            TP += 1
        elif test[i] == 1 and prid[i] == 0:
            FN += 1
    return TP / (TP + FN) if (TP + FN) != 0 else 0

def f1_score(self, test, prid):
    p = self.precision(test, prid)
    r = self.recall(test, prid)
    return (2 * p * r) / (p + r) if (p + r) != 0 else 0

```

KNN scoring

```

In [17]: cm = CM()
acc_knn = cm.accuracy(y_test,y_prid)
precision_knn = cm.precision(y_test,y_prid)
recall_knn = cm.recall(y_test,y_prid)
f1_knn = cm.f1_score(y_test,y_prid)

print(f"Accuracy: {acc_knn:.4f}")
print(f"Precision: {precision_knn:.4f}")
print(f"Recall: {recall_knn:.4f}")
print(f"F1-Score: {f1_knn:.4f}")

```

Accuracy: 0.3100
Precision: 0.6030
Recall: 0.6769
F1-Score: 0.6378

Naïve Bayes code and score

```

In [18]: from sklearn.naive_bayes import MultinomialNB

# Note: MultinomialNB requires non-negative values
# If your TF-IDF has negative values, use GaussianNB instead
nb_classifier = MultinomialNB()
nb_classifier.fit(X_train, y_train)
y_pred_nb = nb_classifier.predict(X_test)

```

```

cm = CM()
acc_nb = cm.accuracy(y_test,y_pred_nb)
precision_nb = cm.precision(y_test,y_pred_nb)
recall_nb = cm.recall(y_test,y_pred_nb)
f1_nb = cm.f1_score(y_test,y_pred_nb)

print(f"Accuracy: {acc_nb:.4f}")
print(f"Precision: {precision_nb:.4f}")
print(f"Recall: {recall_nb:.4f}")
print(f"F1-Score: {f1_nb:.4f}")

```

Accuracy: 0.3850
Precision: 0.6534
Recall: 0.7507
F1-Score: 0.6987

SVM code and score

In [19]: `from sklearn.svm import SVC`

```

svm_classifier = SVC(kernel='linear', C=1.0, random_state=42)
svm_classifier.fit(X_train, y_train)
y_pred_svm = svm_classifier.predict(X_test)

cm = CM()
acc_svm = cm.accuracy(y_test,y_pred_svm)
precision_svm = cm.precision(y_test,y_pred_svm)
recall_svm = cm.recall(y_test,y_pred_svm)
f1_svm = cm.f1_score(y_test,y_pred_svm)

print(f"Accuracy: {acc_svm:.4f}")
print(f"Precision: {precision_svm:.4f}")
print(f"Recall: {recall_svm:.4f}")
print(f"F1-Score: {f1_svm:.4f}")

```

Accuracy: 0.3780
Precision: 0.6677
Recall: 0.6716
F1-Score: 0.6696

KNN, SVM, Naïve Bayes score comparison

```

In [20]: print("\n" + "="*50)
print("PERFORMANCE COMPARISON")
print("="*50)

comparison_df = pd.DataFrame({
    'Classifier': ['KNN', 'Naïve Bayes', 'SVM'],
    'Accuracy': [acc_knn, acc_nb, acc_svm],
    'Precision': [precision_knn, precision_nb, precision_svm],
    'Recall': [recall_knn, recall_nb, recall_svm],
    'F1-Score': [f1_knn, f1_nb, f1_svm]
})

```

```
print(comparison_df.to_string(index=False))

# Find best classifier
best_idx = comparison_df['Accuracy'].idxmax()
best_classifier = comparison_df.loc[best_idx, 'Classifier']
best_accuracy = comparison_df.loc[best_idx, 'Accuracy']
print(f"\nBest Classifier: {best_classifier} with Accuracy: {best_accuracy:.4f}")
```

```
=====
PERFORMANCE COMPARISON
=====
```

Classifier	Accuracy	Precision	Recall	F1-Score
KNN	0.310	0.603030	0.676871	0.637821
Naive Bayes	0.385	0.653367	0.750716	0.698667
SVM	0.378	0.667656	0.671642	0.669643

Best Classifier: Naive Bayes with Accuracy: 0.3850

Score Visualization

```
In [22]: # Plot 1: Accuracy Comparison
plt.figure(figsize=(10, 6))
plt.bar(comparison_df['Classifier'], comparison_df['Accuracy'],
        color=['blue', 'green', 'red'], alpha=0.7)
plt.xlabel('Classifier', fontsize=12)
plt.ylabel('Accuracy', fontsize=12)
plt.title('Accuracy Comparison of Classifiers', fontsize=14, fontweight='bold')
plt.ylim([0, 1])
for i, v in enumerate(comparison_df['Accuracy']):
    plt.text(i, v + 0.02, f'{v:.4f}', ha='center', fontweight='bold')
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

# Plot 2: All Metrics Comparison
fig, ax = plt.subplots(figsize=(12, 6))
x = np.arange(len(comparison_df['Classifier']))
width = 0.2

bars1 = ax.bar(x - 1.5*width, comparison_df['Accuracy'], width, label='Accuracy')
bars2 = ax.bar(x - 0.5*width, comparison_df['Precision'], width, label='Precision')
bars3 = ax.bar(x + 0.5*width, comparison_df['Recall'], width, label='Recall')
bars4 = ax.bar(x + 1.5*width, comparison_df['F1-Score'], width, label='F1-Score')

ax.set_xlabel('Classifier', fontsize=12)
ax.set_ylabel('Score', fontsize=12)
ax.set_title('Performance Metrics Comparison', fontsize=14, fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(comparison_df['Classifier'])
ax.legend()
ax.grid(axis='y', alpha=0.3)
ax.set_ylim([0, 1])
plt.tight_layout()
plt.show()
```

```

# Plot 3: Confusion Matrices
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# KNN Confusion Matrix
cm_knn = confusion_matrix(y_test, y_pred)
sns.heatmap(cm_knn, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=['Negative', 'Neutral', 'Positive'],
            yticklabels=['Negative', 'Neutral', 'Positive'])
axes[0].set_title(f'KNN\nAccuracy: {acc_knn:.4f}', fontweight='bold')
axes[0].set_ylabel('True Label')
axes[0].set_xlabel('Predicted Label')

# Naive Bayes Confusion Matrix
cm_nb = confusion_matrix(y_test, y_pred_nb)
sns.heatmap(cm_nb, annot=True, fmt='d', cmap='Greens', ax=axes[1],
            xticklabels=['Negative', 'Neutral', 'Positive'],
            yticklabels=['Negative', 'Neutral', 'Positive'])
axes[1].set_title(f'Naive Bayes\nAccuracy: {acc_nb:.4f}', fontweight='bold')
axes[1].set_ylabel('True Label')
axes[1].set_xlabel('Predicted Label')

# SVM Confusion Matrix
cm_svm = confusion_matrix(y_test, y_pred_svm)
sns.heatmap(cm_svm, annot=True, fmt='d', cmap='Reds', ax=axes[2],
            xticklabels=['Negative', 'Neutral', 'Positive'],
            yticklabels=['Negative', 'Neutral', 'Positive'])
axes[2].set_title(f'SVM\nAccuracy: {acc_svm:.4f}', fontweight='bold')
axes[2].set_ylabel('True Label')
axes[2].set_xlabel('Predicted Label')

plt.tight_layout()
plt.show()

# =====
# 6. TEST WITH NEW EXAMPLES (Optional)
# =====
print("\n" + "="*50)
print("TESTING WITH NEW EXAMPLES")
print("="*50)

# Example reviews
test_reviews = [
    "This product is absolutely amazing! I love it!",
    "Terrible experience, waste of money.",
    "It's okay, nothing special."
]

tfidf_model = Tfidf()

for review in test_reviews:
    # Convert to TF-IDF
    review_tfidf = tfidf_model.compute_text_tfidf(review)
    review_vector = vectorizer.transform([review_tfidf])

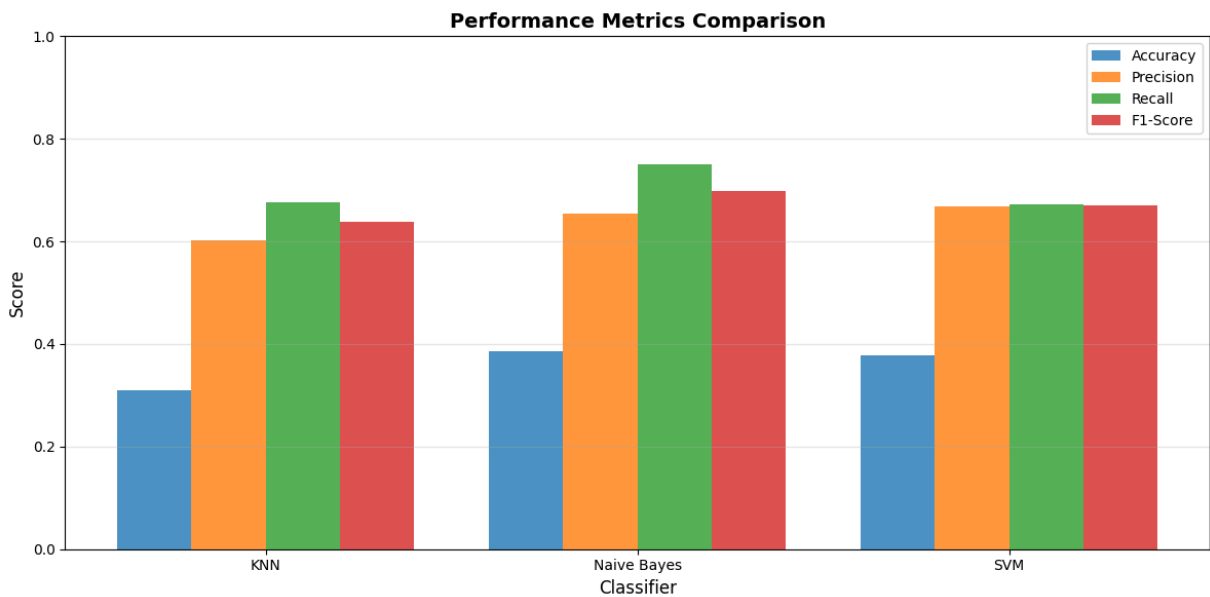
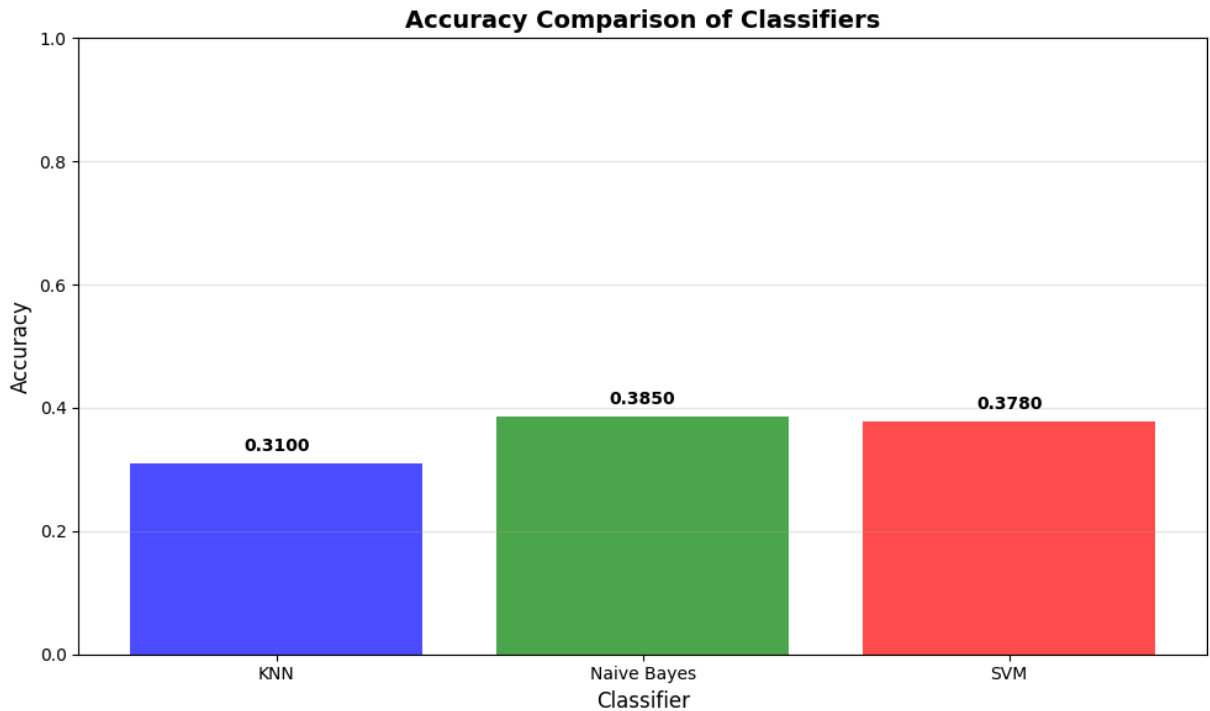
    # Predict with all classifiers

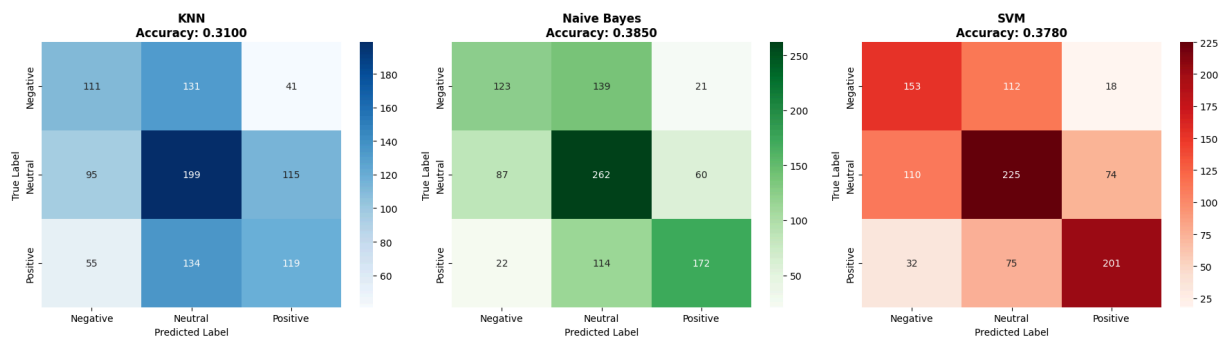
```

```
pred_knn = k.predict(review_vector)[0]
pred_nb = nb_classifier.predict(review_vector)[0]
pred_svm = svm_classifier.predict(review_vector)[0]

sentiment_names = {0: 'Negative', 1: 'Neutral', 2: 'Positive'}

print(f"\nReview: '{review}'")
print(f"   KNN Prediction: {sentiment_names[pred_knn]}")
print(f"   Naive Bayes Prediction: {sentiment_names[pred_nb]}")
print(f"   SVM Prediction: {sentiment_names[pred_svm]}")
```





TESTING WITH NEW EXAMPLES

Review: 'This product is absolutely amazing! I love it!'

KNN Prediction: Neutral

Naive Bayes Prediction: Neutral

SVM Prediction: Neutral

Review: 'Terrible experience, waste of money.'

KNN Prediction: Neutral

Naive Bayes Prediction: Neutral

SVM Prediction: Neutral

Review: 'It's okay, nothing special.'

KNN Prediction: Neutral

Naive Bayes Prediction: Neutral

SVM Prediction: Neutral